

Week 03 Explained Student Worksheet

Text Preprocessing and Tokenization

Name _____ Student ID _____ Group _____

Build the preprocessing function used throughout the retrieval labs. Inspect every transformation so that later index and query results can be explained.

Use this worksheet during the two-hour practical session. Read, predict, run, and explain. The existing lab sheet contains the complete program; this companion explains key ideas and provides space to record your own evidence. Python 3.10 or later is sufficient; no pip packages are needed.

What you should be able to do

- Implement `tokenize(text)` using lowercase conversion and regex.
- Remove stop words using membership in a Python set.
- Construct a sorted collection vocabulary.
- Compare raw text, lowercase text, and token lists.

Understand the concepts

A document is one searchable unit. A token is an occurrence produced by splitting or matching text; repeated words create repeated tokens. In this course, a term is a normalized word retained for indexing. The vocabulary is the set of distinct retained terms across the collection.

The supplied approach is `words = re.findall(r"[a-z]+", text.lower())`. It selects consecutive ASCII letters and skips punctuation and digits. For example, Python, PYTHON becomes two python tokens, while C++ becomes c. It does not stem words, expand contractions, or support all languages.

A stop-word set lists words excluded by the course policy. The supplied set includes common words such as the and and, and also used and uses. Filtering a list preserves repeated content terms and their order. Do not turn tokens into a set before computing term frequencies in later weeks.

Documents and queries must use the same preprocessing. Otherwise a query term may fail to match the representation stored in the index. Keeping the preprocessing function unchanged across weeks makes comparisons interpretable.

Predict before running

In "Python python data", distinguish the number of token occurrences from the number of unique vocabulary terms.

Prepare your terminal

Extract the course and open a terminal inside Week_03_Preprocessing. Keep docs and data beside the Python files. The main demonstration is lab03_solution.py; your exercise starter is lab03_exercises.py. Starters run but deliberately contain unfinished TODO functions.

```
py -3.13 lab03_solution.py
py -3.13 lab03_exercises.py
```

Use py -3 or python instead if that is your working Python 3.10+ command.

Worked example with explanation

This complete small pipeline makes the token/term distinction visible. The full program uses the complete stop-word set from the supplied course files.

```
import re
stop_words = {"the", "and"}
text = "The Python, Python and DATA!"
words = re.findall(r"[a-z]+", text.lower())
tokens = [word for word in words if word not in stop_words]
print(words)
print(tokens)
print(sorted(set(tokens)))
```

Expected output from an executed example

```
['the', 'python', 'python', 'and', 'data']
['python', 'python', 'data']
['data', 'python']
```

How each stage produces the result

1. lower converts the example to a consistent case. This allows Python and python to contribute to the same term.
2. findall with [a-z]+ extracts runs of lowercase ASCII letters. Punctuation separates runs; digits do not become tokens in this required baseline.
3. The comprehension retains each word unless it belongs to stop_words. It preserves token order and repeated occurrences, so python remains twice.
4. set removes duplicates only for vocabulary construction. Sorting that set gives repeatable coordinate order. Do not use the set as the frequency-bearing token list.

Explain one intermediate value in your own words

Read the weekly program

The following excerpt is copied exactly from lab03_solution.py. It depends on the imports, constants, and helper functions in that file. Read it there rather than pasting it into an empty script and expecting the excerpt to run alone.

```
def tokenize(text):  
    """Use the same ASCII-letter tokens as the supplied course files."""  
    words = re.findall(r"[a-z]+", text.lower())  
    return [word for word in words if word not in STOP_WORDS]
```

Code walkthrough

1. The course STOP_WORDS set is shared across the retrieval stages. The short worked example has only two stop words for clarity; the actual lab function uses the full course set.
2. The returned list records occurrences, not only distinct words. Later TF calculations need those repetitions.
3. No stemming, lemmatization, or synonym expansion occurs. learning and learn remain different terms. C++ becomes c, e-mail becomes e and mail, and 2026 contributes no token.
4. Use the same tokenizer for both documents and queries. A search term cannot reliably match a coordinate built using a different normalization policy.

Predict tokens for "Python DATA 2026 e-mail" and identify what information the tokenizer discards.

Connection to the course

The matrix, postings, TF-IDF, and query vectors all inherit this tokenization policy. A preprocessing change requires rebuilding the model before comparing results.

Guided practical activity

Work through the steps in order. Compare your predictions with observations and record at least one specific value per experiment. Keep original baseline data until you have saved the first transcript. Put experiments that change files in a disposable copy.

Step 1 Load the five documents in docs using the course loader.

Keep the five supplied documents unchanged for the baseline trace. Record which collection you used in your transcript.

Step 2 Print each original string and its lowercase form.

Compare case before applying regex. Lowercasing is a normalization step, not stop-word removal.

Step 3 Print the regex matches before filtering stop words.

Separate extraction from filtering so you can identify the step that loses a particular word.

Step 4 Implement `tokenize(text)`, then print its returned list for each document.

Complete the starter tokenizer and use the full course stop-word set. Check the returned type is a list.

Step 5 Accumulate all retained terms in a set and print the sorted vocabulary and its size.

Build the vocabulary with a set, but retain the original token lists for later frequency calculations.

Step 6 Test punctuation, repeated terms, digits, and text consisting entirely of stop words. Record the information lost at each stage.

Test repeated words, punctuation, numbers, and an all-stop-word sentence. Record actual lists rather than saying only that the code works.

Test record

Check to demonstrate	Actual observation	Pass or revise
Mixed case produces consistent lowercase tokens		
Repeated retained terms remain repeated in the token list		
An all-stop-word input returns an empty list		

Record one prediction that changed after running the program and explain why

Independent work and submission

Complete these selected tasks in your own exercise file. Use the existing lab sheet for additional exercises. Do not submit the unchanged starter or simply import a solution in place of implementing the requested function.

- Basic: add a sentence containing repeated Python and data terms. Count tokens and unique terms separately.
- Implementation: finish tokenize in the starter and replace its small practice stop-word set with the full course set.
- Debugging: remove lower() temporarily and explain what the [a-z]+ pattern does to Python and DATA.
- Interpretation: compare tokenization of C++, e-mail, don't, and 2026. Explain the limitations without changing the required pipeline.

Diagnose common errors

TF cannot count repetition later. A set was used too early. Return a list of token occurrences; use a separate set for vocabulary.

Uppercase words disappear or fragment. The [a-z]+ pattern requires lowercase ASCII letters. Apply lower before the regex extraction.

Submit these items

- Your completed tokenize function and a before/after trace for two documents.
- The sorted vocabulary and counts of retained token occurrences and unique terms.
- Three edge-case tests with expected and actual token lists.

Your evidence should identify the input or query, expected result, actual result, and explanation. State any document edits or preprocessing changes so the result can be reproduced. Include this completed worksheet or a clearly numbered digital response.

Exit questions

How does a token occurrence differ from a vocabulary term?

Why should the same tokenizer process documents and queries?

Optional challenge

Add a separate optional tokenizer for Unicode words and compare it with the required ASCII tokenizer on accented and Arabic text. Keep the original implementation as the baseline and explain the effect on vocabulary.

Source for this companion

Based on the supplied Week 03 lab sheet, shipped program, exercise starter, and sample data. The complete main program and expected_output.txt in this folder are the references for checking your baseline run.